Introduction to HTML and CSS?

→ HTML (Hyper text markup Language) is the standard markup language used to create and structure content on web pages. It defines the basic structure of a web page using elements and tags such as headings, paragraphs, links, images, lists, tables, forms and other multimedia content. HTML documents are interpreted by web browsers to display the content in a structured and meaningful manner. HTML provides the structure and semantic organization of a webpage

→ CSS (cascading style sheets) is a stylesheet language used to control the presentation and visual appearance of HTML documents. It is used to define properties such as colors, fonts, backgrounds, spacing, borders, alignment, dimensions, and layouts. CSS also supports responsive design, allowing webpages to adopt to different screen sizes and devices. CSS can be applied using inline, internal or external.

→ HTML and CSS work together to develop webpages. while CSS provides the structure and content, while CSS provides the styling, layout, visual presentation. the combination of both forms the foundation of modern web development and enables the creation of organized, attractive and user-friendly web interfaces.

Difference between MERN stack and FULL stack >

→ MERN stack is a JavaScript-based technology stack used for developing modern web applications. MERN stands for MongoDB, Express.js, React.js, Node.js. MongoDB is used for database management, Express.js handles the backend framework and API's React.js is used to develop frontend user interface, and Node.js provides the server side runtime environment.

→ FULL-stack Development refers to the development of both the frontend and backend of a webapplication along with database integration. A full stack developer works with technologies used for designing the user interface, implementing server-side logic, managing databases.

HTML Components :-

1. HTML Tags - Define the structure and elements of a webpages
2. Headings - used to display titles and headings using <h1> to <h6>
3. paragraphs - used to display text using the <p> tag
4. Links - used to navigate between webpages using <a> tag
5. Images - used to display images using <image> tag
6. Lists - used to create ordered and unordered lists using <ol>, <ul>,
7. Tables - used to organize data in rows and columns
8. Forms - used to collect user input using elements such as textboxes, buttons and checkboxes.
9. Div and span - used to group and organize webpage content
10. Semantic Elements - Elements such as <header>, <nav>, <section>, <article>, <footer> provide meaningful webpage structure.

CSS Components :-

1. Selectors - Select HTML elements to apply styles
2. properties - Defines the visual characteristic of elements
3. values - Specify the setting of a css property
4. Colors - Define text, background, border colors
5. Fonts - control font family, size, style and weight
6. Backgrounds - set background colors and image
7. Borders - Define the border style, width and radius
8. Margin - controls the space outside an element
9. padding - Controls the space inside element
10. layout - Controls the arrangement of elements using flexbox, Grid and positioning.

HTML vs HTML5

HTML	HTML 5
→ Older version of HTML	Latest major version HTML
→ limited Multimedia support	supports audio and video
→ uses external plugins for multimedia	NO external plugs needed.
→ Has fewer semantic tags	provides semantic tags
→ NO <canvas> element	support <canvas>
→ limited web storage	support local storage
→ complex DOCTYPE declaration	simple <!DOCTYPE HTML>

CSS vs CSS3

CSS	CSS3
Earlier version of CSS	Advanced version of CSS
Basic styling features	provides advanced styling features
Limited Animations	supports animation & transition
NO flexible layout systems	supports flexbox and Grid
limited visual effects	Supports shadows, gradients, effects
NO Media queries	supports media queries
Uses a single specification	Divided into different modules

pomodoro Timer , using HTML, CSS and HTML

```
<!DOCTYPE html>
<html>
<head>
    <title> pomodoro Timer </title>
    <style>
        body {
            font-family : Arial;
            background : #f5f5f5;
            text-align : center;
            padding : 50px;
        }
        .timer {
            width : 300 px;
            margin : auto;
            padding : 30 px;
            background : white;
            border-radius : 15px;
            box-shadow : 0 0 10px gray;
        }
        h1 {
            color : #e74c3c;
        }
        .time {
            font-size : 50px;
            margin : 5px;
        }
        button {
            padding : 10px 20px;
            margin : 5px;
            border : none;
            border-radius : 5px;
            background : #e74c3c;
            color : white;
        }
    </style>
</head>
<body>
    <div class = "timer">
        <h1> pomodoro Timer </h1>
        <div class = "time"> 25:00 </div>
        <button> start </button>
        <button> Reset </button>
    </div>
</body>
</html>
```

output :-



22/07/2026

(i) Design an HTML code using audio tags

```
<!DOCTYPE html>
<html>
<head>
    <title> Audio Example </title>
</head>
<body> <h2> My Favorite song </h2>
    <audio controls>
        <source src = "song.mp3" type= audio /mpeg",
        your browser doesnot support the audio tag
    </audio>
</body>
</html>
```

O/P:- Displays an audio player with play, pause and volume controls.

Steps:

Create a new HTML file.
Add the basic HTML structure
Add heading using <h2> tag
Insert the <audio> tag with controls attribute.
Specify the audio file using <source> tag.
Save and open the HTML file in a web browser.
play the audio using displayed controls

(ii) Design a HTML code using video tags.

```
<!DOCTYPE html>
<html>
<head>
    <title> video Example </title>
</head>
<body> <h2> My video </h2>
    <video width="400" controls>
        <source src = 'video.mp4' type = 'video/mp4'>
    </video>
</body>
</html>
```

O/P:- Displays a video player with play, pause, volume and Fullscreen controls.

Steps:-

Create a new HTML file



ave
<h2> tag.
with controls attribute
<source> tag.
+ in web browser.
ayed Controls

write a program using Javascript (arrays, functions)

```
let numbers = [12, 45, 8, 23, 56, 19, 34, 67, 90, 11];
function displayArray(arr) {
    console.log("Array elements:");
    for (let i=0; i<arr.length; i++) {
        console.log(arr[i]);
    }
}
function findSum(arr){
    let sum=0;
    for (let i=0; i<arr.length; i++){
        sum += arr[i];
    }
    return sum;
}
```

O/P:- 12 45 8 23 56 19 34 61 90 11

Sum of array = 365 //

```
function maxArray(arr){
    let max = arr[0];
    for (let i=0; i<arr.length; i++){
        if (arr[i] > max){
            max = arr[i];
        }
    }
```

O/P:- 90

```
function findmin(arr){
    let min = arr[0];
    for (let i=0; i<arr.length; i++) {
        if (arr[i] < min){
            min = arr[i];
        }
    }
    return min;
}
console.log("minimum number:", findmin(number));
```

O/P:- 8

```
function rev(arr){
    return arr.reverse();
}
let fruits = ['apple', 'banana', 'apricot', 'orange',
              'grapes'];
console.log("Reverse fruits:", rev(fruits));
```

O/P:- ['grapes', 'orange', 'apricot', 'banana',
'apple']

11. Develop Dynamic webpage using Javascript.

```
<!DOCTYPE html>
<html>
<head>
    <title> Dynamic Student Management </title>
    <style>
        body {
            font-family : Arial , sans-serif;
            background-color : # f2f2f2;
            margin:0;
        }
        header {
            background-color: # 333;
            color : white;
            text-align : center;
            padding : 20px;
        }
        .container {
            width : 80%;
            margin : 30px auto;
            background-color: white;
            padding : 25px;
            border-radius : 10px;
        }
```

```
        input {
            padding : 10px;
            margin : 5px;
            width : 200px;
        }
        button {
            padding : 10px 20px;
            margin : 5px;
            border : none;
            background-color : # 333;
            color : white;
            cursor : pointer;
            border-radius : 5px;
        }
        button : hover {
            background-color : # 555;
        }
        table {
            width : 100%;
            margin-top : 20px;
            border-collapse : collapse;
        }
        th , td {
            border : 1px solid #ccc;
            padding : 12px;
            text-align : center;
        }
```

```
th {
    background-color: #333;
    color: white;
}
.delete {
    background-color: red;
}
.stats {
    margin-top: 20px;
    font-size: 18px;
    font-weight: bold;
}
#search {
    width: 300px;
}
</style>
</head>
<body>
<header>
    <h1> Student Management System </h1>
    <p> Dynamic webpage using Javascript </p>
</header>
<div class = "container">
    <h2> Add student </h2>
```

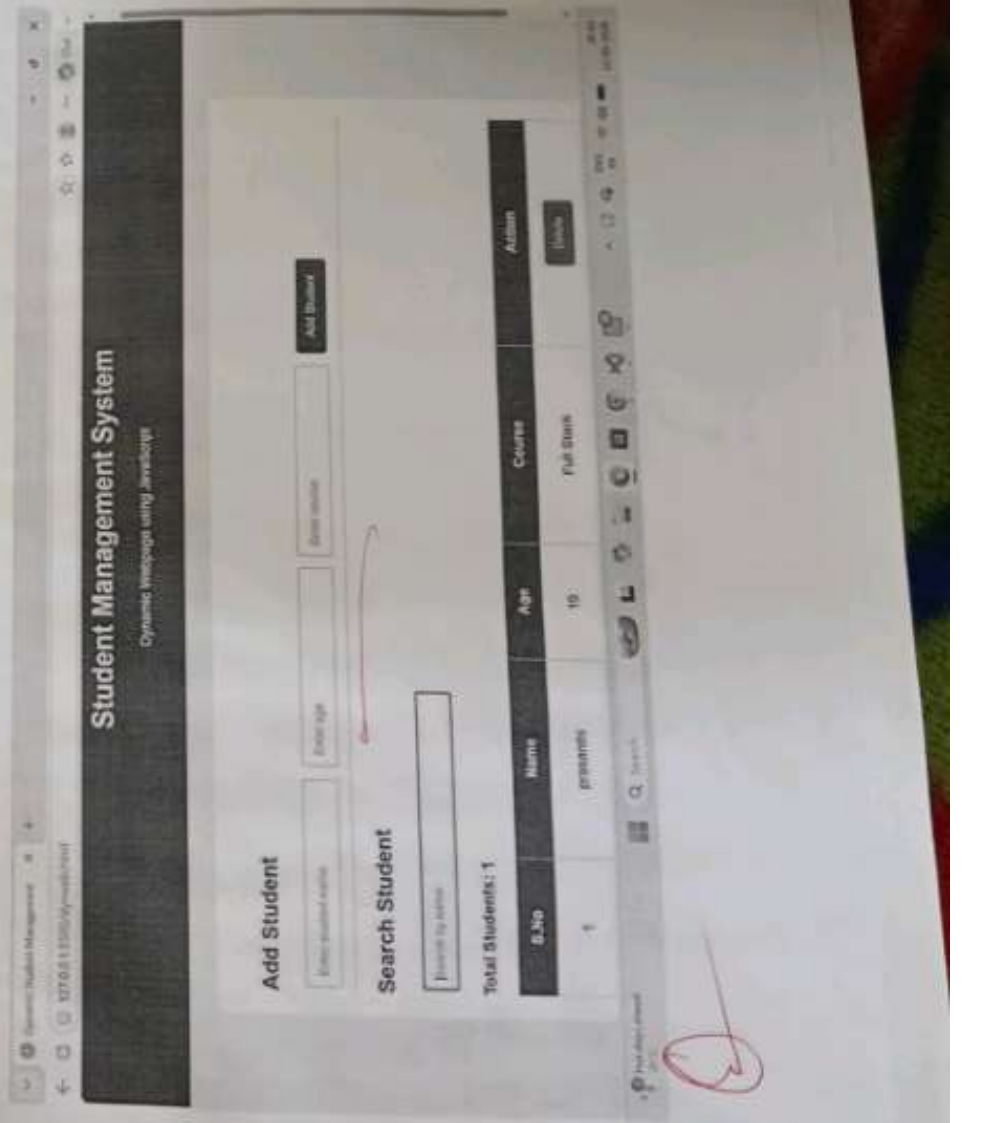
```
<input type = "text" id = "name" placeholder = "Enter student name">
<input type = "number" id = "age" placeholder = "Enter age">
<input type = "text" id = "course" placeholder = "Enter course">
<button onclick = "addStudent()"> Add Student </button>
<hr>
<h2> Search Student </h2>
<input type = "text"
    id = "search"
    placeholder = "search by name"
    onkeyup = "searchStudent()">
<div class = "stats">
    Total Students : <span id = "count">0</span>
</div>
<table>
    <thead>
        <tr>
            <th> S.NO </th>
            <th> Name </th>
            <th> Age </th>
            <th> Course </th>
            <th> Action </th>
        </tr>
```

```
        <tbody id = "student table">
        </tbody>
    </table>
</div>
<script>
    let students = [ ];
    function add Student(){
        let name = document.getElementById("name").
                                                value;
        let age = document.get Element By Id("age").
                                                value;
        let course = document.get Element By Id("course").
                                                value;
        if (name === "" || age === "" || course === ""){
            alert("please enter all details!");
            return;
        }
        let student = {
            name : name,
            age : age,
            course : course
        };
    students.push(student);
    display Students();
```

```
    document.getElementById("name").value = "";
    document.getElementById("age").value = "";
    document.getElementById("course").value = "";
}
function displayStudents(){
    let table = document.getElementById("student table");
    table.innerHTML = "";
    students.forEach(function(student, index){
        let row =
            <tr>
                <td>${index + 1}</td>
                <td>${student.name}</td>
                <td>${student.age}</td>
                <td>${student.course}</td>
                <td>
                    <button class = "delete"
                        onclick = "deleteStudent(${index})">
                        Delete
                    </button>
                </td>
            </tr>
        ;
        table.innerHTML += row;
    });
document.getElementById("count").innerText =
                                        students.length;
```

```
function deleteStudent(index){
    students.splice(index,1);
    displayStudents();
}

function searchStudent(){
    let searchValue =
        document.getElementById("search").value.toLowerCase();
    let rows =
        document.getElementById("studentTable").getElement
                                        ByTagName("tr");
    for(let i=0; i<rows.length; i++){
        let name =
            rows[i].getElementByTagName("td")[1];
        if(name){
            let studentName =
                name.textContent.toLowerCase();
            If(studentName.includes(searchValue)){
                rows[i].style.display = "";
            }
            else{
                rows[i].style.display = "none";
            }
        }
    }
</script>
</body>
```



5) Implement modules in Node.js

modules.js

```
const os = require ("os");
const fs = require ("fs");
const path = require ("path");
const http = require ("http");
const net = require ("net");

console.log ("=== OS MODULE ===");
console.log ("operating system :", os.type());
Console.log ("platform :", os.platform());
console.log ("Architecture :", os.arch());
console.log ("cpu cores :", os.cpus().length);
console.log ("Total Memory :", os.totalmem());
Console.log ("Free Memory :", os.freemem());
console.log ("Home Directory :", os.homedir());

console.log ("\n === PATH MODULE ===");
const filepath = path.join (__dirname, "example.txt");
console.log ("File path :", filepath);
Console.log ("File Name :", path.basename (filepath));
console.log ("Directory :", path.dirname (filepath));
console.log ("Extension :", path.extname (filepath));

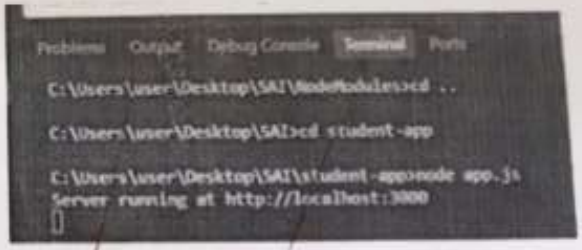
console.log ("\n === FS MODULE ===");
fs.writeFileSync (
    filepath,
    "Hello! This file is created using Node.js FS module"
);
console.log ("File created succesfully.");
const data = fs.readFileSync (filepath, "utf8");
console.log ("File Content :", data);
fs.appendFileSync (
    file path,
    "\n this is additional content.");
console.log ("content added succesfully.");

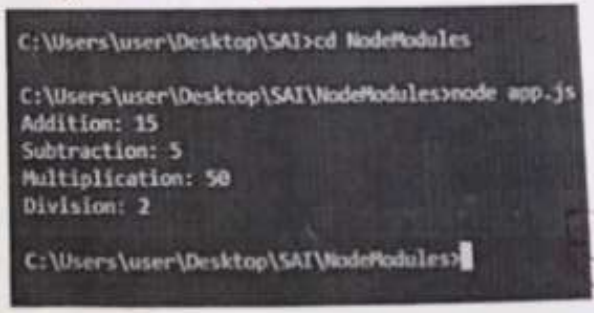
Console.log ("\n === HTTP MODULE ===");
const server = http.createServer ((req, res) => {
    res.writeHead (200, {
        "Content-Type" : "text/html"
    });
    res.write ("<h1> Hello from Node.js! </h1>");
    res.write ("<p> this server is created using HTTP
                module. </p>");
    res.end ();
});
```

```
server.listen(8000, () => {
    console.log("HTTP Server running at http://localhost:8000");
});

console.log("\n === NET MODULE ===");
const netServer = net.createServer((socket) => {
    console.log("Client connected.");
    socket.write(
        "Hello client! Welcome to Node.js NET Server."
    );
    socket.on("data", (data) => {
        console.log("Client says :", data.toString());
    });
    socket.on("end", () => {
        console.log("Client disconnected.");
    });
});
netServer.listen(5000, () => {
    console.log("NET Server running on port 5000");
});
```







6) Develop mini application using Express and Node.js

Server.js:

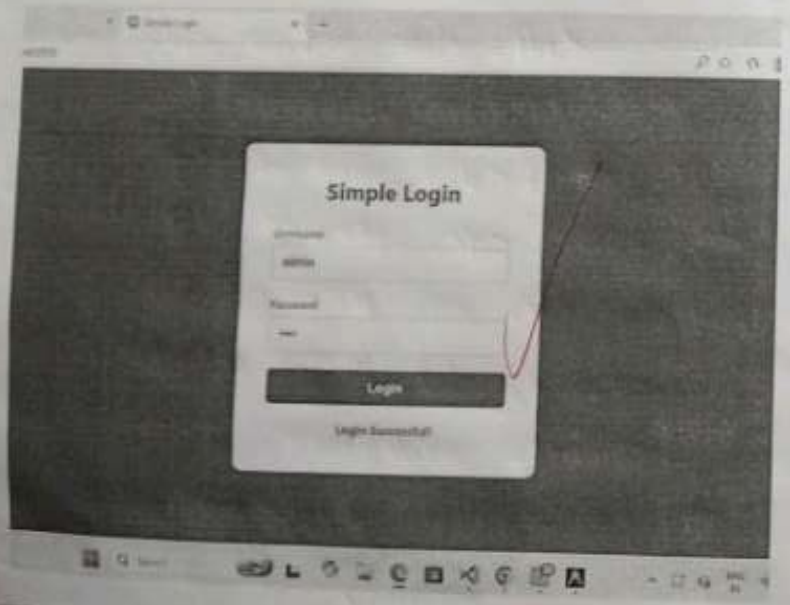
```
const express = require ("express");
const path = require ("path");
Const app = express();
const port = 2000;
app.use (express.json());
app.use (express.urlencoded ({ extended : true }));
app.use (express.static (__dirname));
app.get ("/", (req,res) => {
    res.sendFile (path.join (__dirname, "index.html"));
});
app.post ("login", (req,res) => {
    const username = req.body ? req.body.username:
                                undefined;
    constpassword = req.body ? req.body.password:
                                undefined;
    if(!username || !password){
        return res.status(400).json({
            success: false,
            message: "please enter username and
                                password"
        });
    }
    if (username === "admin" && password === "123"){
        return res.json ({
            success: true,
            message: "Login Successfull!"
        });
    }else{
        return res.status (401).json ({
            success: false,
            message: "Invalid username or password"
        });
    }
});
app.listen(PORT, () => {
    console.log ("server running at http://localhost:
                                ${PORT}");
});
```

index.html:-

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset=UTF-8>
    <title> simple Login </title>
```

```
</head>
<body>
    <div class = "login-card">
        <h1>Simple Login</h1>
        <form onsubmit = "event.preventDefault(); login()">
            <div class = "form-group">
                <label for = "username">username</label>
                <input type = "text" id = "username"
                    placeholder = "Enter username" required>
            </div>
            <div class = "form-group">
                <label for = "password">password</label>
                <input type = "password" id = "password"
                    placeholder = "Enter password" required>
            </div>
            <button type = "submit">Login</button>
        </form>
        <h3 id = "result"></h3>
    </div>
<script>
    async function login(){
        const usernameInput = document.getElementById
                                ("username");
        const passwordInput = document.getElementById
                                ("password");
        const resultEl = document.getElementById("result");
        const username = usernameInput.value.trim();
        const password = passwordInput.value.trim();
        if(!username ||
        if(!username || !password){
            resultEl.innerText = "fill fields";
            resultEl.className = "error";
            return;
        }
        try{
            const response = await fetch("/login", {
                method: "POST",
                headers:{
                    "Content-Type": "application/json"
                },
                body: JSON.stringify({username, password})
            });
            const data = await response.json();
            resultEl.innerText = data.message;
            if(response.ok && data.success !== false){
                resultEl.className = "success";
            }
```

```
else {
    resultEl.className = "error";
    }
} catch (err) {
    resultEl.innerText = "Server Connection error!";
    resultEl.className = "error";
}
</script>
</body>
</html>
```



Experiment -07

07) create a functional component and class component using react.

Aim :- To develop a react application using functional components and classcomponents to display student details, mange state, handle button clicks and dynamically update data on webpage.

functional component :-

A functional component is a javascript function that returns JSX to display the user interface. It uses React hooks such as usestate to manage state and update the webpage dynamically.

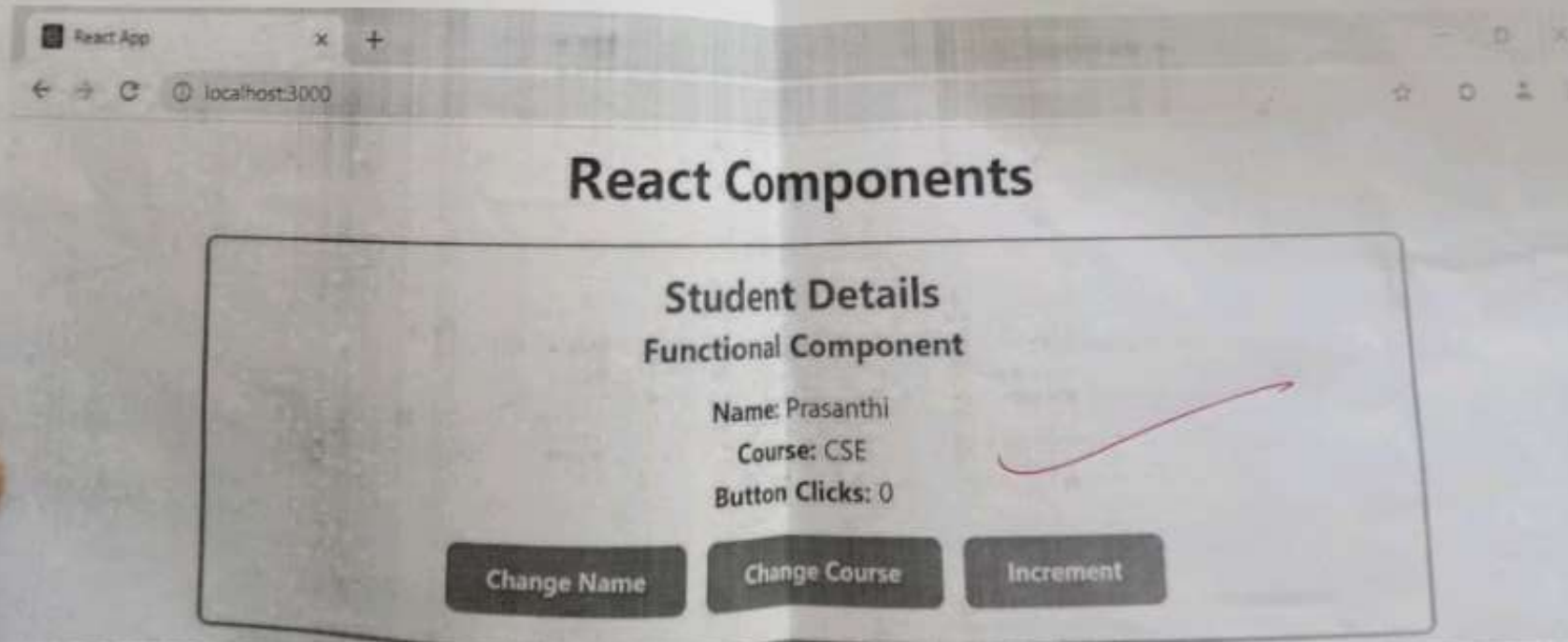
program :-

```
import React, {useState} from
'react';
import
function StudentFunction () {
    const [name, setName] = usestate ("prasanthi");
    const [course, setcourse] = usestate ("csm");
    const [count, setcount] = useState(0);
    const ChangeName = () => {
        setName ("prassu");
```

```
const changeCourse = () => {
    setCourse("CSD");
};
const incrementCount = () => {
    setCount(count+1);
};
return (
  <div style={{
    border: "8px solid blue",
    padding: "20px",
    margin: "20px",
    borderRadius: "10px",
```

```
    <button onClick={incrementCount}>Increment
    </button>
  </div>
);
```

o/p:- when the user clicks the buttons, the name, course and count change dynamically without reloading the webpage.



Class Component :-

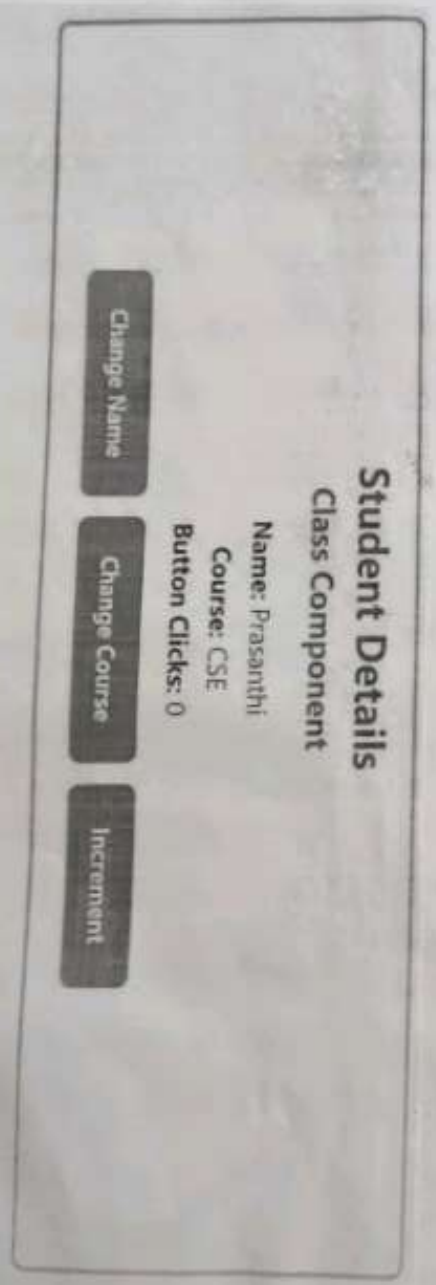
-A class component is an ES6 Javascript class that extends React Component. it uses the render() method to return JSX and manages state using this state and this.setState().

program :-

```
import React, {Component} from 'react';
class StudentClass extends Component{
    constructor(props){
        super(props);
        this.state={
            name: "prasanthi",
            course: "Computer Science",
            count: 0
        };
    }
changeName = () => {
        this.setState({
                name: "prassu"
            });
changeCourse = () => {
        this.setState({
            course: "CSM"
        });
```

```
incrementCount = () => {
    this.setState(prevState => {
        count: prevState.count+1
    });
};
render(){
  return (
    <div style={{
      border: "2px solid green",
      padding: "50px",
      margin: "20px",
      borderRadius: "10px",
      textAlign: "center"
    }}>
    <h1> Student Details </h1>
    <h2> Class Component </h2>
<p> <b> Name: </b> {this.state.name} </p>
<p> <b> Course: </b> {this.state.course} </p>
<p> <b> ButtonClicks: </b> {this.state.count} </p>
<button onClick={this.changeName}>
        ChangeName </button>
<button onClick={this.changeCourse}>
        Change Course </button>
<button onClick={this.incrementCount}> increment
        </button>
</div>
```

O/P :-



8)

Experiement -05

Implement http methods using JSON server.

Theory :- JSON server is a tool used to create a fake REST API using a JSON file - it allows us to perform HTTP operations without developing a backend server from Scratch.

HTTP methods are used to communicate between a client and a server.

- GET — Retrieves data from the server
- POST — Adds new data to the server
- PUT — Replaces an existing record.
- PATCH — updates selected fields of a record
- DELETE — Removes a record from the server

a. create a JSON server :-

Step1 :- Initialize the project

Open the Vscode terminal and execute the following commands

```
mkdir JSON server
Cd JSON server
npm init -Y
npm install json server.
```

Step 2:- create db.json

create a file named db.json and add the following student data

```
{ "students": [
    { "id": "1",
      "name": "prasanthi",
      "course": "CSM",
      "marks": "90"
    },
    { "id": "2"
      "name": "likhitha",
      "course": "CSM",
      "marks": "90"
    },
    { "id": "3"
      "name": "sahithi",
      "course": "IT",
      "marks": "89"
    }
```

Step 3:- Start the JSON server

For the current JSON server version, execute:

npx. json. server db.json

The server start at:

http://localhost:3000

The student records can be accessed using

http://localhost:3000/students

2) Implementation of http methods:-

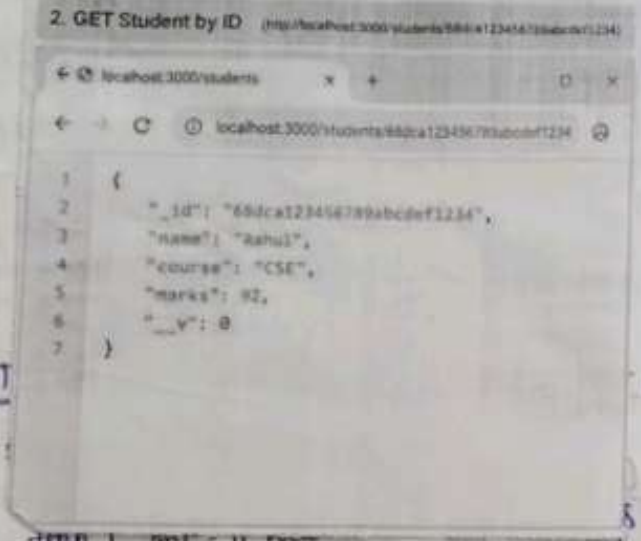
a) GET Method:- The GET method is used to retrieve student records from JSON server

code:-

```
fetch('http://localhost:3000/students')
  .then(response => response.json())
  .then(data => {
      console.log(data);
  })
  .catch(error => {
      console.log(error);
  });
```

O/P:-

Outpu



B post

new

code: fetch ('http://localhost:3000/students', {

method: 'POST',

headers: {

'content-type': 'application/json'

},

body: JSON.stringify ({

name: 'Rahul',

course: 'CSE',

marks: 92

})

})

.then (response => response.json())

.then (data => {

console.log (data);

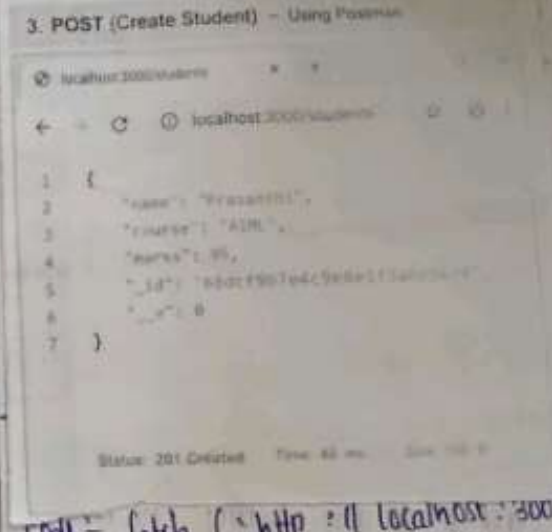
})

• catch (error => {

console.log (error);

});

3. POST (Create Student) - Using Postman



to update an d data.

code: fetch ('http://localhost:3000/students/1', {

method: 'PUT',

headers: {

'content-type': 'application/json'

},

body: JSON.stringify ({

name: 'prasanthi',

course: 'AIML',

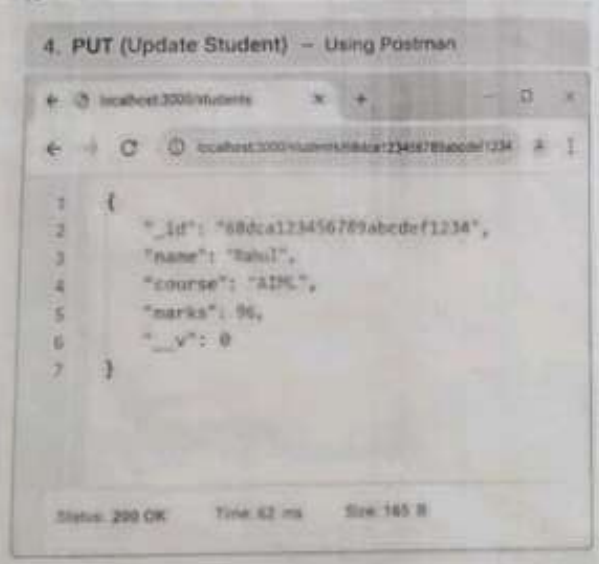
marks: 95

})})

.json())

```
.then (data => {
    console.log (data);
})
.catch (error => {
    console.log (error);
```

output:-



D. PATCH

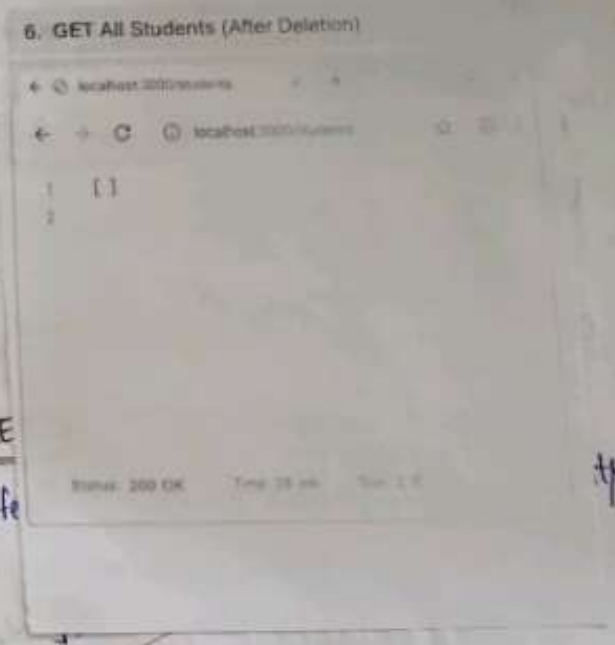
update record

code:-

```
method: 'PATCH',
headers: {
'content-TYPE': 'application/json'
},
body: JSON.stringify ({
  marks: 96
})
```

```
.then (response => response.json ())
.then (data => {
    console.log (data);
})
.catch (error => {
    console.log (error);
});
```

output:-



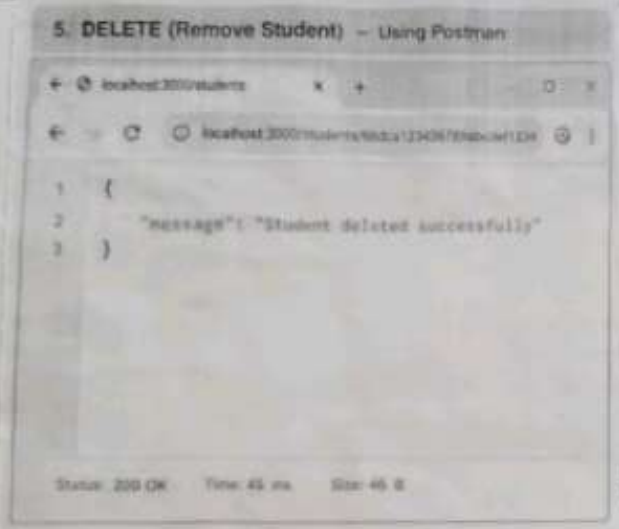
E. DELE

code:-

```
.then (response => {
  if (response.ok) {
    console.log ("student deleted successfully");
  }})
.catch (error => {
    console.log (error)
```

output:-

5. DELETE (Remove Student) – Using Postman



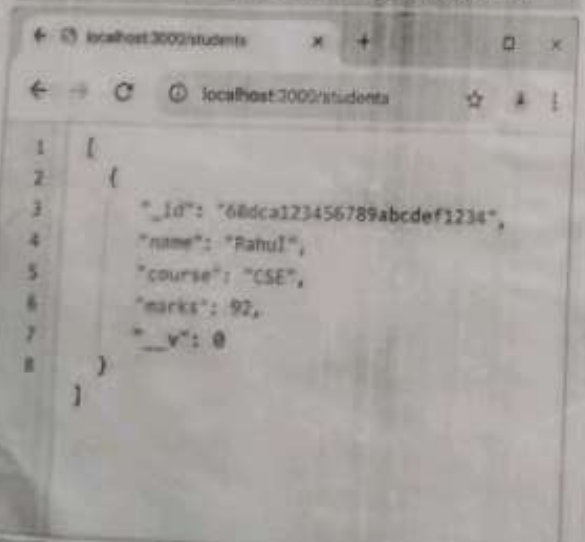
All

DEL

Student will be:

Output:-

1. GET All Students (http://localhost:3000/students)



Experiment no - 9:-

9) Develop CRUD operations using MongoDB.

Theory:- MongoDB is a NoSQL database that stores data in flexible, JSON-like documents, CRUD operations are used to manage data in database.

MongoDB is an object Data Modelling (ODM) library for MongoDB and Node.js it helps define schemas interact with collections and perform database operations.

1. Installation and project setup:

```
mkdir MongoDB - CRUD
cd MongoDB - CRUD
npm init -y
npm install express mongoose
```

program:- server.js

```
const express . require ('express');
const mongoose : require ('mongoose');
const . app = express();
const . port = 3000;
app . use ( express . json());
```

```
mongoose . connect ('mongodb://127.0.0.1:27017/
                    college)
. then() → console.log ("MongoDB connected")
. catch (err → console. log (err));

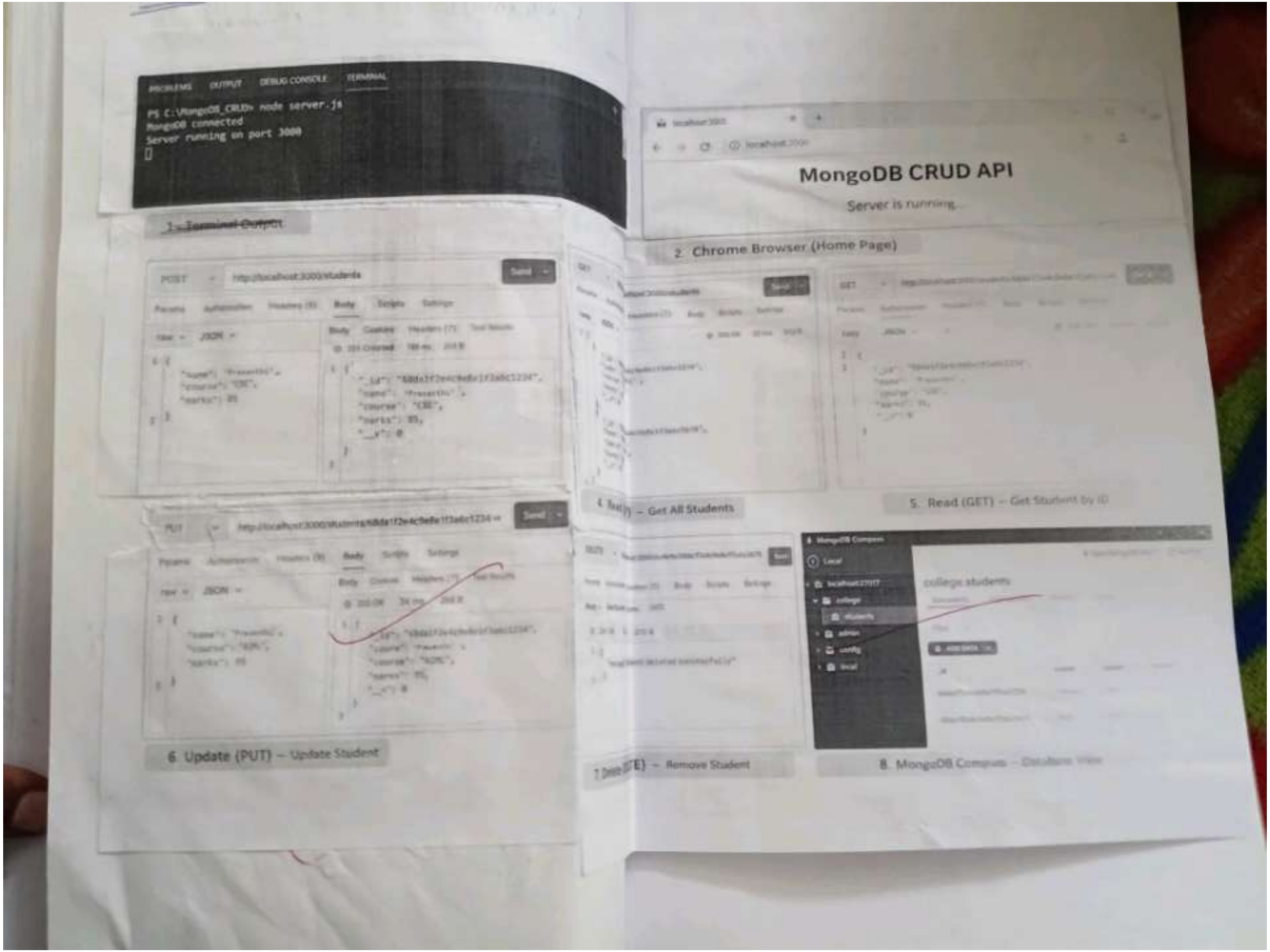
// create student
const studentschema = new
mongoose . schema ({
    name : {
      type : string,
      required : true
    },
   course : {
     type : string,
     required : true
   },
   marks : {
     type : Number
     required : true
   }
});

// create student model
const student :
  mongoose . model ('student', studentschema);
```

```
// CREATE : Add
app. post ('/students', asyn(req, res) => {
  try {
    const student = new Student (req. body);
    const savedstudent = await student save();
res. status (201) . json (Savedstudent);
} catch (error) {
    res. status (400). json ({message error:
                              message});
  }
});
// READ : Get all students
app .get ('/students', async (req, res) → {
    try {
      const students = await
        student . find ();
        res. status (500) . json ({message : error:
                                   message});
}});
// READ : Get student by ID
app . get ('/students/ :id ', async (req, res) => {
    try {
      const student = await
        student. findById ( req. params. id);
```

```
if( !student) {
    return res.status (404).json({
      message: "student not found"
    });
  }
  res.status (200).json (student)
} catch (error) {
  res.status (400).json {{message:
                          error.message}};
    }
});
//UPDATE
app.PUT ('/students/:id', async (req, res) =>{
  try {
    const student = await
    Student findById and Update(
        req.params.id,
        req.body,
      { new : true , runValidators: true}
    );
if(! student) {
  return res.status (404).json({
    message: "student not found"
  });
}
```

```
  res.status(200).json (student);
} catch (error){
  res.status (400).json ({message =
                          error.message});
  }
});
//DELETE
app.delete ('/students/:id', async (req,res) =>{
    try{
      const student = await student,
                findBy Id and Delete (
    req.params.id);
if (!student) {
  return res.status (404).json({
    message: "student not found"
  });
}
res.status (200).json({
  message: "student deleted succesfully" });
} catch (error) {
  res.status (400).json ({message:
                          error.messag e });
    }
});
```


PROBLEMS
OUTPUT
DEBUG CONSOLE
TERMINAL
PS C:\MongoDB_CRUD> node server.js
MongoDB connected
Server running on port 3000
MongoDB CRUD API
Server is running
2. Chrome Browser (Home Page)
6. Update (PUT) – Update Student
8. MongoDB Compass – Database View